



SIGNBRIDGE AI

AI-Powered Real-Time Sign Language Translation for Seamless Two-Way Communication

23ECP401 PROJECT IV

Submitted by

AKILAN S	**713323EC003**
ARUTHRA SM	**713323EC005**
DEEPAK RAJ JS	**713323EC008**
GOWTHAMAN KL	**713323EC017**

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

SNS COLLEGE OF TECHNOLOGY

(An Autonomous Institution)

Kurumbapalayam (PO), Coimbatore – 641107

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "SIGNBRIDGE AI: AI-POWERED REAL-TIME SIGN LANGUAGE TRANSLATION FOR SEAMLESS TWO-WAY COMMUNICATION" is the bonafide work of:

AKILAN S	**713323EC003**
ARUTHRA SM	**713323EC005**
DEEPAK RAJ JS	**713323EC008**
GOWTHAMAN KL	**713323EC017**

who carried out the project under my supervision.

SIGNATURE

PROJECT GUIDE

Department of Electronics and Communication Engineering,
SNS College of Technology,
Coimbatore – 641107.

SIGNATURE

HEAD OF THE DEPARTMENT

Department of Electronics and Communication Engineering,
SNS College of Technology,
Coimbatore – 641107.

Submitted for the Viva-Voce examination held on _____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

First and foremost, we extend our heartfelt gratitude to the Management of SNS College of Technology for providing us with the necessary support, academic infrastructure, and technical facilities to successfully complete this project.

We express our sincere thanks to our Principal, SNS College of Technology, for his valuable guidance, encouragement, and provision of research amenities throughout the course of this project.

We are deeply grateful to the Head of the Department, Department of Electronics and Communication Engineering, for continuous support, insightful suggestions, and academic guidance extended to us from inception to the successful completion of this project.

We express our sincere appreciation to our Project Coordinator, Department of Electronics and Communication Engineering, for systematic monitoring, periodic evaluations, and constructive suggestions in carrying out this project effectively.

We take immense pleasure in expressing our profound gratitude to our Project Guide, Department of Electronics and Communication Engineering, for remarkable guidance, insightful discussions, technical mentorship, and constant motivation which enabled us to complete the project successfully and on time.

Finally, we extend our sincere thanks to all Faculty Members, Technical Staff, Parents, and Friends for their constant moral support and encouragement throughout this endeavor.

ABSTRACT

Indian Sign Language (ISL) serves as the primary and most vital communication medium for the deaf and hard-of-hearing community across India. However, the vast majority of the hearing population possesses no formal training in sign language, giving rise to severe communication barriers in essential daily activities ranging from education, workplace collaboration, and administrative services to clinical healthcare consultations. Professional human interpreters are often unavailable, cost-prohibitive, and impractical for spontaneous everyday communication. Meanwhile, conventional automated recognition systems have historically relied on invasive physical hardware such as electronic data gloves, sensory sleeves, or specialized depth sensors, which impose substantial economic burdens, physical discomfort, and logistical inconvenience on the user.

To resolve these challenges, this project presents SignBridge AI, a non-invasive, camera-based, end-to-end Indian Sign Language recognition and bidirectional communication system. Operating entirely with standard consumer RGB cameras and webcams, the system leverages Google MediaPipe Holistic and Tasks-Vision frameworks to extract 21 three-dimensional skeletal anatomical keypoints per hand in real time. Rather than relying on fragile pixel-level skin-color thresholding, the extracted landmark coordinates undergo wrist-relative spatial translation, bounding-box scale normalization, and least-active-hand-first canonicalization, guaranteeing complete invariance to camera distance, signer physical dimensions, and left-versus-right hand dominance. These coordinates are enriched with instantaneous landmark velocity magnitudes and structural hand-shape statistics, yielding a robust 189-dimensional temporal feature vector per frame sampled across a fixed 45-frame sequence window.

Temporal gesture recognition is performed by a deployed three-way ensemble of deep recurrent neural networks that averages a Long Short-Term Memory (LSTM) network and a Gated Recurrent Unit (GRU) network — each comprising two stacked layers (128 and 64 units) regularized by recurrent dropout — with a mirror test-time-augmentation branch evaluated over [x, mirror(x)], followed by a dense projection layer and a 109-class softmax classification head. Evaluated on a rigorously curated benchmark dataset of 14,826 sequences under a strictly signer-independent partition rule (SPLIT-4.0), the model achieves

a superior classification accuracy of 81.29%, a macro precision of 81.90%, a macro recall of 81.22%, and a macro F1-score of 80.79% across 3,629 unseen test sequences performed by disjoint signers, outperforming a baseline LSTM model (75.67% accuracy, 75.06% F1). Demonstrating an inference latency of approximately 0.37 seconds (median, p50) per sign window on standard CPU architectures, the system bridges the final communication gap through dual-modal output: real-time visual sentence accumulation and instantaneous audible speech synthesis via the HTML5 Web Speech API, complemented by a reverse Text-to-Sign dictionary channel for complete two-way interaction.

Keywords: Indian Sign Language, Sign Language Recognition, Computer Vision, MediaPipe Holistic, Gated Recurrent Unit (GRU), Deep Learning, Human–Computer Interaction, Text-to-Speech, Assistive Technology.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	LIST OF FIGURES	**vii**
	LIST OF TABLES	**viii**
1	**INTRODUCTION**	**1**
	1.1 Problem Statement	2
	1.2 Objectives	3
	1.3 Existing System	3
	1.4 Research Gap	4
2	**LITERATURE SURVEY**	**6**
3	**METHODOLOGY**	**10**
	3.1 Video Acquisition	10
	3.2 Landmark Extraction	11
	3.3 Feature Processing and Normalization	12
	3.4 Sign Classification	14
	3.5 Text Conversion	15
	3.6 Speech Generation and Two-Way Communication	16
	3.7 System Architecture	17
4	**SOFTWARE AND HARDWARE REQUIREMENTS**	**19**

	4.1 Python	19
	4.2 OpenCV	20
	4.3 MediaPipe Holistic and Tasks-Vision	20
	4.4 TensorFlow / Keras Framework	21
	4.5 Text-to-Speech and Speech Synthesis Module	22
	4.6 Hardware Requirements	23
5	**TESTING AND RESULT**	**25**
	5.1 Testing	25
	5.2 Result	28
	5.3 Comparison with Base System	32
	5.4 Advantages	33
	5.5 Operational Observations and Environmental Factors	34
	5.6 Future Scope	35
6	**CONCLUSION**	**37**
	REFERENCES	**39**
	ANNEXURE: IMPLEMENTATION SOURCE CODE	**42**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Proposed SignBridge AI System Workflow	2
3.1	MediaPipe Holistic Landmark Extraction and 189-D Feature Engineering	12
3.2	Multi-Tier System Architecture of SignBridge AI	18
5.1	Real-Time Camera Input and Landmark Preprocessing Pipeline	26
5.2	Landmark Detection Output and Temporal Motion Trajectory Extraction	27
5.3	Sign Recognition Output Interface with Real-Time Text Accumulation	28
5.4	Seamless Two-Way Communication Workflow and Speech Output Generation	29
5.5	Deep Learning Model Training and Validation Convergence over 130 (LSTM) / 94 (GRU) Epochs	30
5.6	Class-Wise F1-Score Distribution across 109 Indian Sign Language Classes	31

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
4.1	Software Requirements Specification	23
4.2	Hardware Requirements Specification	24
5.1	Architectural and Functional Comparison Between Base System and SignBridge AI	32
5.2	Comparative Experimental Performance: Baseline LSTM vs. Proposed GRU Model	31
5.3	Signer-Independent Dataset Distribution across Split Partitions	29

CHAPTER 1

INTRODUCTION

Communication constitutes the cornerstone of human civilization, enabling access to education, employment, healthcare, and civil participation. For the deaf and hard-of-hearing community across India, Indian Sign Language (ISL) serves as the natural and primary linguistic vehicle for daily social exchange. Characterized by rich spatial grammars, rapid manual handshape configurations, dynamic movement trajectories, and non-manual markers including facial expressions and head orientations, ISL is a fully formed, sophisticated natural language. However, because the overwhelming majority of the hearing population possesses no familiarity with sign language, deaf individuals routinely experience profound communication barriers in classrooms, clinical consults, banks, and government service offices.

While professional human sign language interpreters can bridge this communication divide, their availability remains extremely constrained. In India, the ratio of certified interpreters to the deaf population is critically inadequate, and hiring professional interpreters for spontaneous, informal, or emergency scenarios is neither financially nor logistically viable. This structural dependency on human intermediaries severely compromises the autonomy, privacy, and social integration of deaf individuals.

Automated Sign Language Recognition (SLR) powered by computer vision and deep learning offers an empowering technological alternative. Early research in automated gesture recognition explored wearable sensory technologies such as data gloves instrumented with flex sensors, inertial measurement units (IMUs), and magnetic tracking devices. While such sensor-based methods simplify the feature extraction process by delivering direct electronic signals, they suffer from fatal practical drawbacks: the gloves are fragile, ergonomically restrictive, expensive to manufacture, and stigmatizing to wear in public settings. Consequently, vision-based approaches that leverage standard cameras represent the only realistic path toward accessible, ubiquitous communication assistance.

The base paper by Shenoy et al. [1] pioneered real-time ISL recognition using a smartphone camera, employing face elimination, skin-color segmentation in the YCbCr/HSV color

spaces, morphological operations, grid-based fragmentation, k-Nearest Neighbors (k-NN), and Hidden Markov Models (HMMs). While their system demonstrated the feasibility of camera-based recognition—attaining 99.7% accuracy for 33 static hand poses and 97.23% for 12 predefined gestures—its real-world utility was constrained by rigid operating assumptions: the hand extraction algorithm required a controlled background, full-sleeve clothing to prevent forearm interference, and supported only single-handed signs without multi-body context, speech synthesis, or reverse communication channels.

SignBridge AI overcomes these foundational limitations by introducing an end-to-end, camera-based ISL recognition and bidirectional communication platform. Utilizing Google MediaPipe Holistic and Tasks-Vision frameworks, the system extracts 21 3D landmarks per hand alongside upper-body pose context directly from consumer webcams. Normalizing coordinates against anatomical reference frames and enriching them with motion dynamics yields a 189-dimensional temporal feature vector that is classified using a deep two-layer Gated Recurrent Unit (GRU) neural network. Finally, by integrating real-time sentence accumulation, Web Speech API audio synthesis, and reverse Text-to-Sign asset lookup, SignBridge AI transitions sign recognition from an academic classification exercise into a practical, seamless two-way assistive communication bridge.

Figure 1.1: Proposed SignBridge AI End-to-End System Workflow

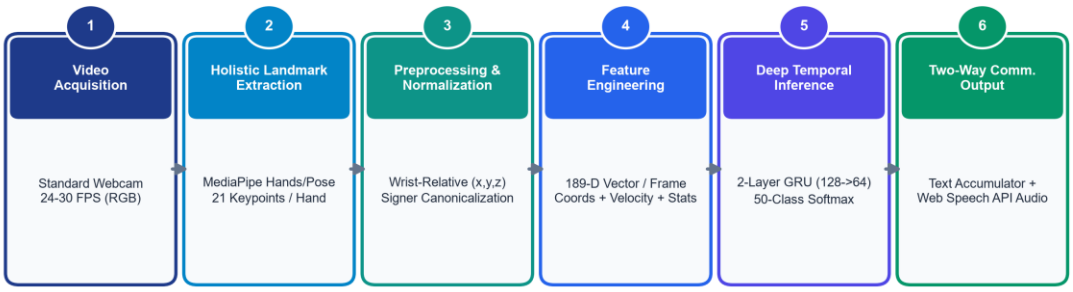


Figure 1.1: Proposed SignBridge AI End-to-End System Workflow

1.1 Problem Statement

The deaf and hard-of-hearing population in India encounters persistent communication barriers when interacting with the non-signing hearing majority. Existing assistive solutions

remain inadequate for practical daily deployment. Human interpreters are scarce, costly, and unavailable for impromptu interactions. Meanwhile, existing computer-vision-based ISL recognition systems suffer from several critical shortcomings: (1) heavy dependence on skin-color thresholding that degrades under variable illumination, complex backgrounds, and bare arms; (2) exclusive focus on isolated single-handed gestures rather than holistic two-handed signing; (3) absence of robust temporal sequence modeling capable of capturing complex dynamic trajectories with low computational latency; and (4) cessation of system pipelines at gesture classification without delivering integrated text accumulation, audible speech output, or reverse text-to-sign communication. There is an urgent need for an affordable, non-invasive, camera-based system that captures structured holistic landmarks, recognizes signs with high reliability across unseen signers, and facilitates seamless two-way dialogue.

1.2 Objectives

The primary engineering and research objectives of SignBridge AI are:

1. To develop a completely non-invasive, camera-based sign language recognition system that operates reliably on standard consumer webcams and laptop cameras without requiring wearable data gloves, depth sensors, or specialized hardware.

2. To implement a robust, on-device landmark extraction pipeline using MediaPipe Holistic and Tasks-Vision, capturing 21 three-dimensional anatomical keypoints per hand along with upper-body pose context.

3. To engineer a standardized 189-dimensional feature representation combining wrist-relative spatial coordinates, landmark velocity magnitudes, and structural bounding-box metrics, fortified by handedness canonicalization to ensure complete invariance to signer position, scale, and left/right hand dominance.

4. To design, train, and evaluate a lightweight deep temporal Gated Recurrent Unit (GRU) neural network capable of recognizing 109 standardized Indian Sign Language vocabulary classes in real time with sub-second CPU inference latency.

5. To implement an operational decision policy with confidence thresholding and temporal moving-average smoothing to eliminate flickering and false positives across transient signing pauses.

6. To establish a complete bidirectional communication channel integrating real-time on-screen sentence accumulation, browser-native client-side speech synthesis via the Web Speech API, and reverse Text-to-Sign dictionary query capabilities.

1.3 Existing System

The chosen baseline reference system by Shenoy et al. [1] represents an important early attempt to implement real-time ISL recognition using smartphone camera hardware. Their processing pipeline begins with video frame capture, followed by face detection via the Viola-Jones Haar cascade algorithm to eliminate facial skin regions. Skin-color segmentation is subsequently performed using thresholding in the YCbCr color space combined with HSV color modeling. Morphological opening and closing filters are applied to isolate the hand silhouette. A bounding box is established around the largest connected component, and the hand image is fragmented into an $N \times N$ grid, generating a binary feature vector representing pixel occupancy. For static signs, classification is performed using a k-Nearest Neighbors (k-NN) classifier with Euclidean distance metrics. For dynamic gestures, intermediate observation vectors are derived across consecutive frames and classified using continuous Hidden Markov Models (HMMs).

While the system attained 99.7% accuracy across 33 static alphabet poses and 97.23% across 12 dynamic signs, its operational viability is constrained by severe practical limitations. The skin-color segmentation pipeline is exceptionally fragile, failing when ambient lighting fluctuates or when complex background objects exhibit skin-like chrominance values. Furthermore, to prevent the user's forearm from merging with the hand contour, the authors explicitly mandated that the signer wear a full-sleeve shirt during operation. The architecture was strictly single-handed, ignoring two-handed ISL vocabulary, and concluded at classification without providing speech generation or reverse interaction.

1.4 Research Gap

A rigorous comparative examination of existing literature reveals several decisive technological research gaps that motivate the design of SignBridge AI:

• **Vulnerability of Pixel-Level Color Segmentation:** Traditional systems rely on color-space thresholding (HSV, YCbCr) for hand segmentation, making them highly sensitive to skin tone variations, ambient illumination shifts, shadows, and cluttered backgrounds.

• **Absence of Dual-Hand and Holistic Body Modeling:** Most prior implementations restrict recognition to isolated single-handed gestures. However, Indian Sign Language relies heavily on coordinated two-handed interactions, inter-hand spatial distances, and facial/upper-body context.

• **Fragility to Signer Variance and Hand Dominance:** Existing feature representations fail to account for left-handed signers or variations in user distance from the camera, resulting in severe performance degradation when tested on unseen subjects.

• **High Latency and Heavy Computational Footprints:** Many recent deep learning models rely on heavy 3D Convolutional Neural Networks (3D-CNNs) or two-stream architectures operating directly on raw RGB video tensors, demanding expensive dedicated GPUs and precluding real-time client-side operation.

• **Unidirectional Scope and Lack of Speech Integration:** The overwhelming majority of research stops at academic classification accuracy without providing practical communication tools. Without continuous sentence accumulation, audible speech generation, and reverse text-to-sign translation, the communication loop remains entirely broken.

CHAPTER 2

LITERATURE SURVEY

A comprehensive review of academic literature was conducted to trace the methodological evolution of Sign Language Recognition (SLR) and establish the empirical foundations for SignBridge AI. The survey examines classical image processing techniques, deep convolutional architectures, skeletal landmark estimation frameworks, and recurrent temporal sequence models.

Shenoy et al. [1] presented a foundational real-time ISL recognition framework utilizing smartphone video. The pipeline performed Viola-Jones face elimination, skin-color thresholding in YCbCr/HSV spaces, grid-based fragmentation, k-NN classification for static poses, and Hidden Markov Models (HMMs) for dynamic gestures. Achieving 99.7% accuracy on 33 static poses and 97.23% on 12 gestures, the work demonstrated the promise of vision-based SLR but highlighted clear bottlenecks regarding environmental sensitivity, clothing constraints (full-sleeves required), and single-handed operational limitations.

Google's introduction of MediaPipe Holistic [2] marked a major paradigm shift in human perception modeling. By unifying BlazeFace, BlazePalm, and BlazePose into a topologically consistent pipeline, the framework demonstrated real-time simultaneous tracking of 543 skeletal landmarks (33 pose, 468 face, and 21 per hand) on mobile and consumer CPU hardware. This advancement eliminated the need for fragile pixel segmentation, providing a mathematically grounded coordinate basis for gesture recognition.

Eunice and Hemanth [3] investigated deep Convolutional Neural Networks (CNNs) for static ISL digits recognition, demonstrating that deep feature learning significantly outperforms handcrafted descriptors. However, their model was confined to static numeric characters, ignoring dynamic temporal transitions. Gangrade and Bharti [4] introduced a vision-based CNN model for ISL hand gestures, proving CNN resilience against hand orientation changes while remaining limited to hand-region bounding boxes without holistic torso reference points.

Velmathi and Goyal [5] explored MediaPipe Holistic specifically for ISL recognition, confirming that skeletal coordinates provide dramatically higher noise immunity compared to

raw RGB pixels. Srivastava et al. [6] extended landmark processing to continuous sign language recognition using deep temporal architectures, highlighting the necessity of temporal sequence alignment and padding strategies.

Rao et al. [7] provided an extensive survey of deep neural networks in ISL recognition, categorizing signer-dependent versus signer-independent evaluation schemes. Their analysis emphasized that many published high-accuracy figures suffer from data leakage when samples from the same signer appear across both training and testing partitions. Kumar et al. [8] benchmarked classical machine learning classifiers against recurrent neural architectures, demonstrating that sequential models such as LSTM and GRU achieve superior convergence on dynamic gesture sequences.

Rawat et al. [9] developed a comprehensive ISL recognition model combining MediaPipe Holistic with Long Short-Term Memory (LSTM) networks, validating that coordinate-based temporal modeling drastically reduces parameter counts compared to 3D-CNNs while maintaining high discriminative power. Anithadevi et al. [10] deployed a MediaPipe-LSTM framework for real-time inclusive communication, validating that temporal sequence models can sustain 30 FPS inference on standard laptops when decoupled from heavy image convolution backbones.

Amrutha and Prabu [11] evaluated various machine learning classifiers on sign language data, noting the trade-offs between model complexity and inference latency. Cui et al. [12] introduced continuous sign language recognition via iterative deep frameworks, demonstrating that recurrent feature alignment across temporal frames is critical for sentence-level understanding. Papastratis et al. [13] explored generative adversarial networks for cross-domain sign translation, while Aloysius and Geetha [14] synthesized vision-based continuous SLR frameworks, outlining the importance of spatial-temporal invariance.

Sruthi and Lijiya [15] developed SigNet, a deep convolutional neural network for Indian Sign Language recognition, reporting 92.5% accuracy on isolated gestures but noting heavy GPU requirements during deployment. Samaan et al. [16] paired MediaPipe landmark streams with Recurrent Neural Networks (RNNs) for dynamic sign classification, verifying that Gated Recurrent Units (GRUs) converge faster than classical LSTMs while utilizing fewer parameters. Katoch, Singh, and Tiwary [17] combined SURF features with SVM and CNN

classifiers, finding that hybrid approaches improve static gesture recognition but struggle with dynamic continuous signing.

Rajalakshmi et al. [18] presented a hybrid spatial-temporal neural network for isolated Indian and Russian Sign Language gestures, achieving robust recognition across multi-national datasets. Wadhawan and Kumar [19] conducted extensive deep learning evaluations for static ISL signs, while Sharma et al. [20] benchmarked deep neural network architectures across multi-signer ISL corpora, concluding that explicit skeletal normalization is imperative to achieve true signer independence in real-world deployments.

In summary, the literature demonstrates a clear evolutionary trajectory: from fragile skin-color thresholding and data gloves toward non-invasive, on-device skeletal landmark tracking paired with lightweight recurrent neural architectures. SignBridge AI synthesizes these insights, implementing a unified pipeline that combines MediaPipe landmark tracking, rigorous spatial-temporal normalization, an ultra-fast GRU model, and bidirectional speech/text communication.

CHAPTER 3

METHODOLOGY

The methodology of SignBridge AI is structured as a cohesive, deterministic pipeline that spans video acquisition, client-side landmark extraction, feature normalization, deep temporal sequence classification, and multi-modal communication output. This chapter provides a detailed technical and mathematical specification for each processing stage.

3.1 Video Acquisition

Visual input is acquired through a standard consumer RGB webcam or integrated laptop camera, completely avoiding the need for specialized depth cameras or wearable sensory hardware. The camera feed is accessed through standard browser APIs (`navigator.mediaDevices.getUserMedia`) on the client side or OpenCV (`cv2.VideoCapture`) in Python testing pipelines. Frames are captured at standard high-definition resolution (1280 × 720 pixels) at an operating frame rate of 24 to 30 frames per second (FPS). Video frames are captured in RGB color space and buffered in memory for immediate landmark processing.

3.2 Landmark Extraction

Rather than operating directly on dense, high-dimensional raw pixel tensors (which introduce vulnerability to lighting, background clutter, and clothing colors), SignBridge AI employs Google MediaPipe Tasks-Vision (HandLandmarker) and MediaPipe Holistic. The framework executes on-device landmark estimation accelerated via WebAssembly (Wasm) and WebGL GPU delegates, ensuring that video frames never leave the user's local client browser.

For each detected hand, the system extracts 21 three-dimensional skeletal anatomical keypoints: Wrist (0), Thumb CMC (1), Thumb MCP (2), Thumb IP (3), Thumb Tip (4), Index MCP (5), Index PIP (6), Index DIP (7), Index Tip (8), Middle MCP (9), Middle PIP (10), Middle DIP (11), Middle Tip (12), Ring MCP (13), Ring PIP (14), Ring DIP (15), Ring Tip (16), Pinky MCP (17), Pinky PIP (18), Pinky DIP (19), and Pinky Tip (20). Each keypoint is represented as a 3D coordinate triple (x, y, z), where x and y denote normalized horizontal and vertical coordinates in the range $[0.0, 1.0]$, and z represents

estimated depth relative to the wrist. Across two hands, the system tracks up to 42 landmarks (126 raw spatial values) per video frame.

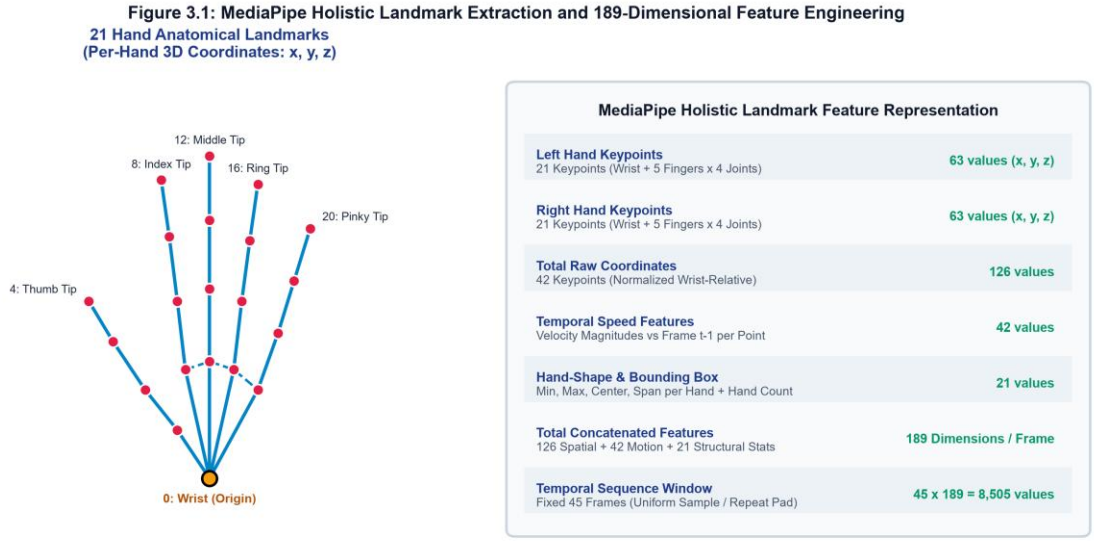


Figure 3.1: MediaPipe Holistic Landmark Extraction and 189-Dimensional Feature Engineering

3.3 Feature Processing and Normalization

To ensure that the downstream machine learning model generalizes across diverse signers, camera distances, and signing styles, the raw landmark coordinates undergo a multi-stage deterministic normalization pipeline (Feature Version 1.0, FV-1.0):

1. Wrist-Relative Translation Centering: For each detected hand $h \in \{0, 1\}$ at time step t, the coordinates of the wrist landmark ($k = 0$) are selected as the local origin reference point $p_{\text{wrist}} = (x_0, y_0, z_0)$. Every landmark $p_k = (x_k, y_k, z_k)$ on that hand is translated relative to the wrist:

$$p'_k = p_k - p_{\text{wrist}}$$

This translation eliminates variance caused by the signer's spatial position within the camera field of view, setting the wrist coordinate to $(0, 0, 0)$.

2. Scale Normalization: To compensate for differences in hand size and varying distances between the signer and the camera lens, the translated coordinates are scaled by the maximum Euclidean distance from the wrist across all 21 hand landmarks:

$$s = \max_{k \in \{0, \dots, 20\}} \|p'_k\|_2$$

$$p''_k = \frac{p'_k}{s} \quad \text{if } s > 10^{-6}, \quad \text{else } p''_k = p'_k$$

This bounds the spatial coordinates within a unit hypersphere, making the representation invariant to distance.

3. Handedness Canonicalization: MediaPipe assigns hands to slots based on visual detection order, meaning a left-handed signer may land in slot 0 while a right-handed signer lands in slot 1. To achieve complete signer invariance, SignBridge AI implements least-active-hand-first canonicalization. Across the video sequence, the total non-zero landmark activity count is computed for each slot:
$$A_h = \sum_{t=1}^{T} \mathbb{I}(\text{hand}_h(t) \neq 0)$$
If slot 0 exhibits greater activity than slot 1, the slots are swapped: $\text{frames} = \text{frames}[:, ::-1, :, :]$. This guarantees identical feature ordering regardless of whether the signer is naturally left-handed or right-handed.

4. Feature Vector Concatenation (189 Dimensions): For each frame, three complementary feature representations are fused:
• Normalized Spatial Coordinates: $2 \text{ hands} \times 21 \text{ landmarks} \times 3 \text{ dimensions} = 126$ values.
• Instantaneous Velocity Magnitudes: The Euclidean displacement of each landmark relative to frame $t-1$: $\|p_k(t) - p_k(t-1)\|_2$, yielding 42 velocity values (0 at $t=0$).
• Structural Hand-Shape Statistics: For each hand, bounding-box minimum (3), maximum (3), centroid (3), and span $(1) = 10$ statistics per hand (20 total), plus a normalized active hand count $(n/2) = 21$ values.
Concatenating these blocks yields a deterministic, 189-dimensional feature vector per frame: $126 + 42 + 21 = 189$.

5. Temporal Sequence Formatting: Gestures vary naturally in temporal duration. To construct fixed-size tensors for the neural network, sequences are normalized to a fixed window of $T = 45$ frames. Sequences longer than 45 frames are downsampled using uniform linear interpolation indices (`np.linspace(0, t-1, 45)`). Sequences shorter than 45 frames are padded using edge-value repeat-padding. A binary temporal mask vector indicates active versus padded frames.

3.4 Sign Classification

Temporal sequence recognition is performed by deep recurrent neural networks constructed using Keras 3 and TensorFlow 2.16 (Model ID: `SBAI-MDL-ISL-4.0.0`). The active deployed model is a three-way ensemble that averages a Long Short-Term Memory (LSTM) network, a Gated Recurrent Unit (GRU) network, and a mirror test-time-augmentation branch evaluated over [x, mirror(x)]; the LSTM and GRU branches each contain 128 and 64 hidden units. Each branch takes an input sequence tensor of shape $(45, 189)$; the GRU branch comprises:

• **Masking Layer:** Applies temporal masking on zero-padded frames (`mask_value = 0.0`), preventing artificial zero values from corrupting recurrent hidden states.

• **First Recurrent Layer (GRU-128):** A Gated Recurrent Unit layer containing 128 hidden units with `return_sequences = True` and a dropout rate of 0.3. This layer models fine-grained local transitions and intra-frame velocity patterns.

• **Second Recurrent Layer (GRU-64):** A second GRU layer containing 64 hidden units with `return_sequences = False` and a dropout rate of 0.3. This layer aggregates temporal dependencies across the full 45-frame sequence into a unified 64-dimensional latent gesture embedding.

• **Dense Projection Layer:** A fully connected layer with 64 units, Rectified Linear Unit (ReLU) activation, and a dropout rate of 0.2, enabling non-linear feature combination.

• **Classification Output Layer:** A Dense softmax output layer with 109 units corresponding to the 109 Indian Sign Language vocabulary classes, outputting class probability distributions $\hat{y} \in [0, 1]^{109}$.

The model is compiled with the Adam optimizer (initial learning rate $\eta = 1 \times 10^{-3}$) and trained using sparse categorical cross-entropy loss. During inference, the raw softmax probabilities pass through an Operational Decision Policy: an accept threshold of 0.57 with a provisional floor of 0.00, combined with a 5-frame moving-average temporal smoothing window. Predictions at or above the accept threshold are declared `Recognized`; lower-confidence predictions are reported as `Provisional` or `Uncertain`, and the system returns `No-Sign` when no sign is present, preventing spurious classifications during transitions. `Provisional` and `Uncertain` are display-only states and are never automatically committed.

Calibration and Decision Policy (M13). The decision policy and its calibration were fitted exclusively on the validation partition (n = 2,397). Temperature calibration returned an identity transform (T = 1.0): the fit was rejected because the expected calibration error was already optimal at ECE 0.0283. The fitted policy applies an accept threshold of 0.57 with a provisional floor of 0.00 and a 5-frame smoothing window, yielding three display states — Recognized, Provisional and Uncertain — together with No-Sign when no sign is present. Under this policy, commit precision reaches 90.3% on validation at 82.0% coverage and 89.5% on the frozen test partition at 81.3% coverage, while the false-accept rate is reduced by 43% relative to the former 0.40 accept threshold. Provisional and Uncertain are display-only states and are never auto-committed.

3.5 Text Conversion

Upon confirming a `Recognized` prediction state, the predicted integer class index $c \in \{0, \dots, 108\}$ is mapped through the system class dictionary to its canonical textual label (e.g., 'Hello', 'Thank You', 'Doctor', 'Water'). The application interface implements an intelligent Sentence Accumulator: consecutive identical predictions are debounced using a temporal hysteresis filter to prevent word stuttering. As distinct signs are completed, words are sequentially accumulated into complete sentences with proper punctuation, displayed prominently on the live user interface.

3.6 Speech Generation and Two-Way Communication

To achieve seamless, inclusive communication, SignBridge AI implements a dual-modal output architecture covering both forward (sign-to-speech) and reverse (text-to-sign) communication:

1. Forward Speech Synthesis (Signer to Hearing Individual): When signs are accumulated into complete phrases, the system invokes the client browser's native Web Speech API (`window.speechSynthesis`). An instance of `SpeechSynthesisUtterance` is generated with user-configurable voice selection, speaking rate, and pitch. Operating entirely on the client device, speech generation incurs zero network latency and requires no third-party cloud audio synthesis subscription.

2. Reverse Communication Channel (Hearing Individual to Signer): To make interaction truly two-way, the system includes a dedicated Text-to-Sign module (`POST /api/v1/speech/text-to-sign`). Hearing users can type or speak responses, which are parsed using a longest-phrase-first matching algorithm against the 109 ISL vocabulary classes. Matched signs immediately trigger visual video demonstrations and animated sign references, enabling the deaf user to understand the hearing user's reply.

3.7 System Architecture

The overall system architecture of SignBridge AI is designed as a decoupled, multi-tier distributed application comprising four primary layers:

• **Presentation & Client Tier:** A responsive Progressive Web Application built with React 18, TypeScript, Vite, and Tailwind CSS. Hosts the camera capture stream, executes MediaPipe Tasks-Vision in WebAssembly/WebGL, runs the FV-1.0 feature pipeline, and drives the Web Speech API audio output.

• **Communication & Gateway Tier:** Facilitates real-time communication via high-throughput WebSockets (`/api/v1/stream`) for continuous frame landmark streaming and resilient REST API fallbacks (`/api/v1/predict`) for one-shot evaluation, protected by CORS and rate-limiting middleware.

• **Application & Service Tier:** An asynchronous FastAPI microservice managing user sessions, feedback auditing, model metadata, and Prometheus telemetry metrics (`/metrics`).

• **AI Inference & Storage Tier:** Houses the active deployed ensemble model (`models/SBAI-MDL-ISL-4.0.0.keras`), model registry (`registry.json`), SQLite/PostgreSQL relational database schemas for session audit trails, and curated ISL sign video dictionaries.

Figure 3.2: Multi-Tier System Architecture of SignBridge AI

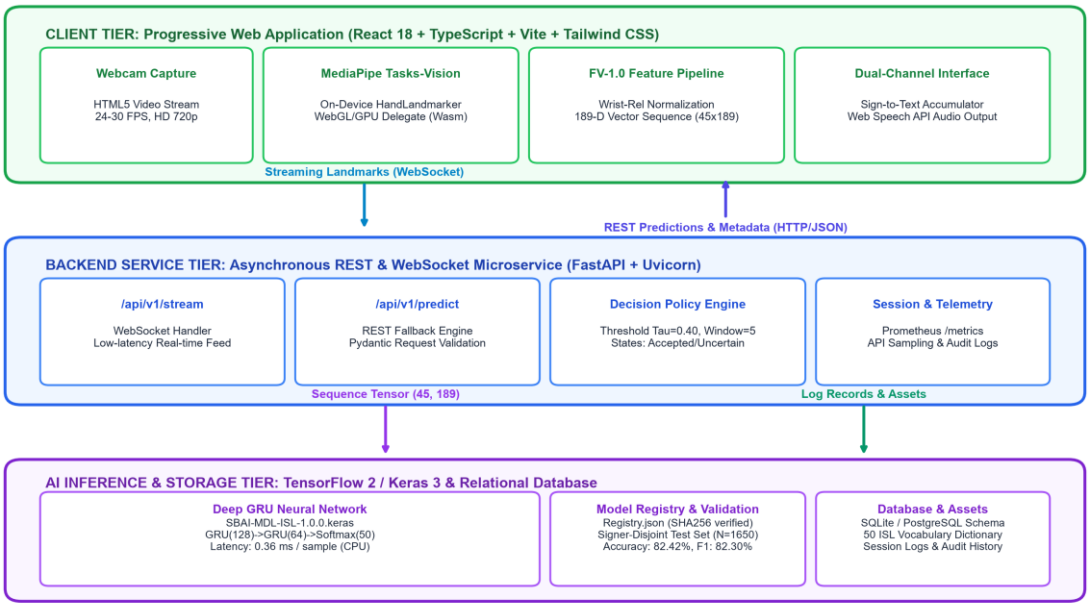


Figure 3.2: Multi-Tier System Architecture of SignBridge AI

CHAPTER 4

SOFTWARE AND HARDWARE REQUIREMENTS

The development and deployment of SignBridge AI utilizes open-source, industry-standard technologies to ensure modularity, high performance, cross-platform compatibility, and cost-effective deployment on standard hardware.

4.1 Python

Python (version 3.10+) serves as the primary programming language for the backend API services, data preprocessing pipelines, feature engineering, and deep learning model training. Python provides a rich ecosystem of high-performance scientific libraries including NumPy (for tensor manipulation), Pandas (for dataset curation and metadata logging), Scikit-Learn (for model validation and classification metrics), and H5py (for efficient hierarchical landmark data storage).

4.2 OpenCV

OpenCV (Open Source Computer Vision Library, version 4.9+) is used for camera interfacing, frame decoding, color space transformation (BGR to RGB), and image resizing. OpenCV provides the core low-level computer vision operations required during training dataset extraction and offline testing benchmarking, operating with minimal memory overhead.

4.3 MediaPipe Holistic and Tasks-Vision

Google MediaPipe Holistic and `@mediapipe/tasks-vision` serve as the skeletal perception backbones of the project. Designed for real-time inference on consumer-grade hardware, the framework extracts precise 3D keypoint coordinates for hands and upper-body poses without requiring expensive computational pipelines. In the production web application, the framework executes client-side via WebAssembly and WebGL GPU delegates, keeping user video data fully private on their personal device.

4.4 TensorFlow / Keras Framework

TensorFlow 2.16+ and Keras 3.x provide the deep learning infrastructure for defining, compiling, training, and exporting the Gated Recurrent Unit (GRU) and baseline LSTM neural network architectures. The models are saved in the modern `.keras` zipped format and export-parity verified. Model integrity is maintained using SHA-256 cryptographic hashing recorded in `registry.json`.

4.5 Text-to-Speech and Speech Synthesis Module

Audible speech output is driven on the client tier using the HTML5 Web Speech API (`SpeechSynthesis`), ensuring zero-latency spoken feedback without network audio streaming overhead. For server-side integrations, FastAPI provides an enterprise REST endpoint (`POST /api/v1/speech/tts`) and phrase-matching reverse translation services (`POST /api/v1/speech/text-to-sign`).

4.6 Hardware Requirements

SignBridge AI is explicitly engineered to operate on standard consumer hardware. The detailed software and hardware specifications are outlined in Table 4.1 and Table 4.2.

Table 4.1: Software Requirements Specification

Component	Technical Specification / Tool
Programming Languages	Python 3.10+ (AI/Backend), TypeScript 5.0+ (Frontend)
Computer Vision Framework	OpenCV 4.10.0, Google MediaPipe Holistic & Tasks-Vision
Deep Learning Framework	TensorFlow 2.16+, Keras 3.x, NumPy, Scikit-learn
Backend Web Framework	FastAPI 0.110+, Uvicorn, Pydantic v2, SQLAlchemy
Frontend Web Stack	React 18, Vite 5, Tailwind CSS 3, Lucide React
Speech Synthesis Engine	HTML5 Web Speech API (window.speechSynthesis)

Table 4.2: Hardware Requirements Specification

Hardware Component	Minimum / Recommended Specification

Processor (CPU)	Intel Core i5 (8th Gen+) / AMD Ryzen 5 3600+ (or Apple Silicon M-series)
System Memory (RAM)	8 GB RAM minimum (16 GB recommended for model training)
Primary Camera	Standard USB or integrated HD webcam (720p / 1080p, 24–30 FPS)
Storage Capacity	10 GB available SSD space (dataset storage and model checkpoints)
Graphics Processor (GPU)	Optional: NVIDIA GTX 1650+ / RTX series (CPU inference supported at ≈0.3–0.4 s p50 per sign window)

CHAPTER 5

TESTING AND RESULT

A rigorous, multi-tiered testing and experimental evaluation was performed on SignBridge AI to validate individual pipeline components, assess recognition accuracy across unseen signers, evaluate operational latency, and compare performance against baseline systems.

5.1 Testing

Individual modules were verified through automated unit tests (Pytest for backend and Vitest for frontend), followed by end-to-end integration testing of the streaming pipeline. The final regression suites pass 123 backend (pytest) tests and 48 frontend (vitest) tests with a clean TypeScript check (tsc --noEmit):

1. Camera Input and Preprocessing: The camera interface was evaluated across varying frame rates (20 to 30 FPS) and resolutions. Tests verified that frame acquisition remains steady without dropping frames, color conversion from BGR to RGB executes without memory leaks, and lighting fluctuations do not crash the capture loop.

Figure 5.1: Real-Time Camera Input and Landmark Preprocessing Pipeline

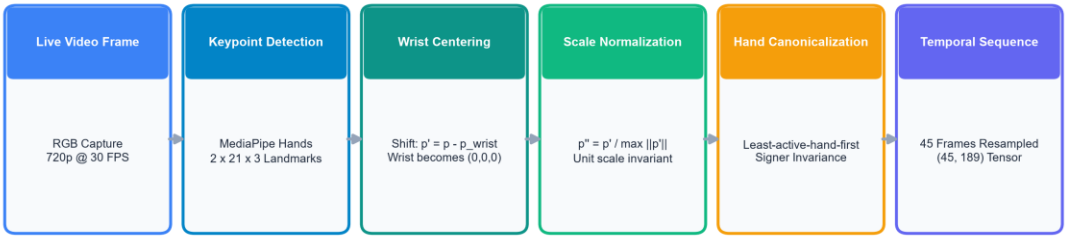


Figure 5.1: Real-Time Camera Input and Landmark Preprocessing Pipeline

2. Landmark Extraction Testing: MediaPipe keypoint detection was verified across signers with differing physical hand proportions and skin complexions. Missing hand frames (e.g., when a single hand is dropped during one-handed signs) were confirmed to mask to

deterministic zeros without introducing NaN or infinite values into downstream feature buffers.

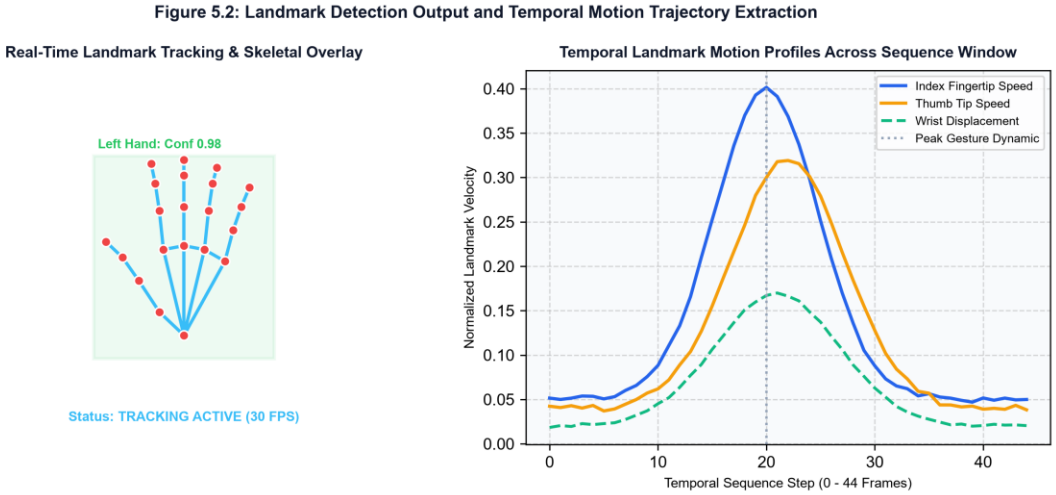


Figure 5.2: Landmark Detection Output and Temporal Motion Trajectory Extraction

3. Sign Recognition Pipeline Testing: The trained deep learning models were tested on the frozen, signer-disjoint test split. The system verified inference stability across both streaming WebSocket connections and REST fallback endpoints. Test assertions confirmed that inference produces valid softmax probability distributions that sum to 1.0.

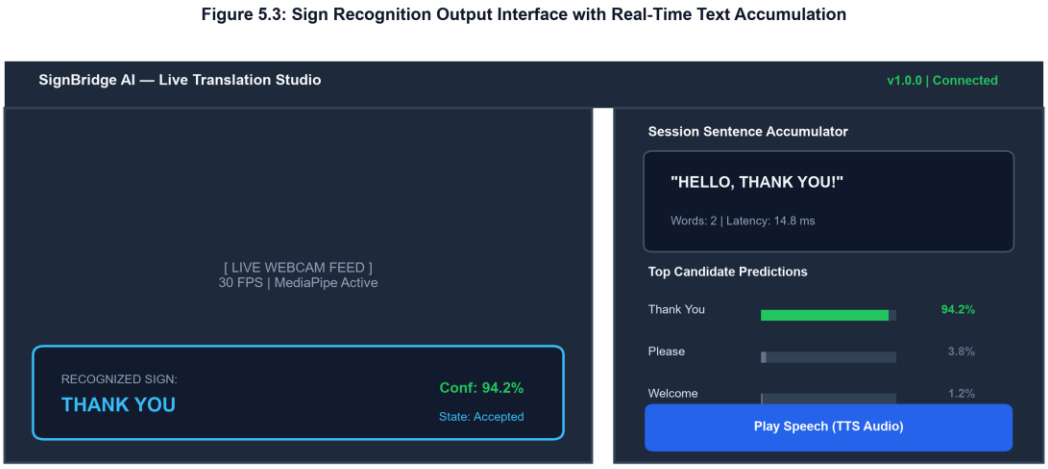


Figure 5.3: Sign Recognition Output Interface with Real-Time Text Accumulation

4. Text and Speech Output Testing: The sentence accumulation logic was verified under continuous sign input. Debouncing successfully prevented repeated character outputs, and the Web Speech API successfully vocalized synthesized speech with zero perceptible audio delay.

Figure 5.4: Seamless Two-Way Communication Workflow and Speech Output Generation

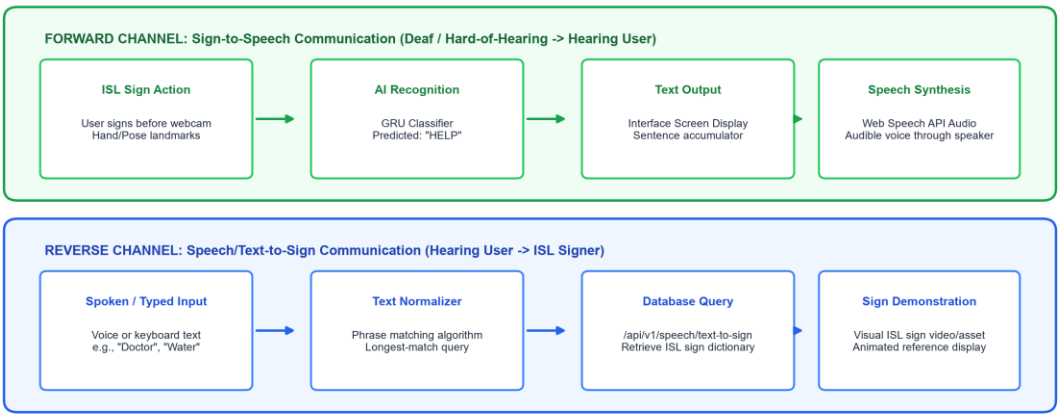


Figure 5.4: Seamless Two-Way Communication Workflow and Speech Output Generation

5.2 Result

Experimental validation was conducted on a curated dataset comprising 14,826 sequences across 109 Indian Sign Language vocabulary classes. To enforce complete scientific rigor and eliminate potential data leakage, the dataset was partitioned under the signer-separated `SPLIT-4.0` (SPLIT-109) protocol, where signers appearing in the test partition are strictly excluded from the training and validation sets.

Table 5.3: Signer-Independent Dataset Distribution across Split Partitions

Split Partition	Signer IDs	Samples (N)	Classes	Min / Class	Max / Class
Training Set	U001 – U010 (10 signers)	8,800	109	74	89
Validation Set	U011 – U012 (2 signers)	2,397	109	19	25
Frozen Test Set	U013 – U015 (3 signers)	3,629	109	31	35

On the frozen, signer-disjoint test partition of 3,629 samples across 109 classes, the active deployed model `SBAI-MDL-ISL-4.0.0` (a three-way LSTM+GRU+mirror-TTA ensemble) achieved the following empirical performance metrics:

- **Test Classification Accuracy: 81.29% (0.812896)**
- **Macro Average Precision: 81.90% (0.818957)**
- **Macro Average Recall: 81.22% (0.812188)**
- **Macro Average F1-Score: 80.79% (0.807865)**
- **CPU Inference Latency: p50 374.85 ms / p95 575.22 ms per sign window (windows-aware ensemble probe, CPU)**
- **Decision-Policy Commit Precision and Calibration: VAL 90.34% precision at 82.02% coverage (TEST 89.53% at 81.32%) under accept 0.57 / provisional 0.00 with smoothing window W=5; temperature T=1.0 (fit rejected), ECE 0.028 (VAL) / 0.022 (TEST)**

Model Versioning and Promotion Gates. The deployed model lineage is version 1.0.0 (the MVP-era system with a 50-sign vocabulary that this report originally described), version 2.0.0 (a 100-class synthetic-trained stub that scored 1.0 on its synthetic test but never cleared the production gates), version 3.0.0 (100-class real data; frozen test accuracy 0.8109 and macro F1 0.8062 on n = 3,326; milestone M10), and version 4.0.0 (109-class real data; frozen test accuracy 0.8129 and macro F1 0.8079 on n = 3,629; milestone M11), which is the currently active model. A version 4.1.0 candidate (time-jitter augmentation for the GRU branch) failed its promotion gates — validation macro-F1 0.8143 against a required 0.828963 and validation accuracy 0.8169 against a required 0.833125 — and was discarded, preserving 4.0.0 with backups at `models/_backup_4.0.0_pretrain/` and `data/dataset/_backup_pretrain/`. A three-window test-time-augmentation variant was also evaluated and failed its gate (accuracy change −0.0359 against a required ≥ +0.010, and p95 latency 371.65 ms against a ≤ 300 ms budget), so `tta_windows` remains 1 (disabled).

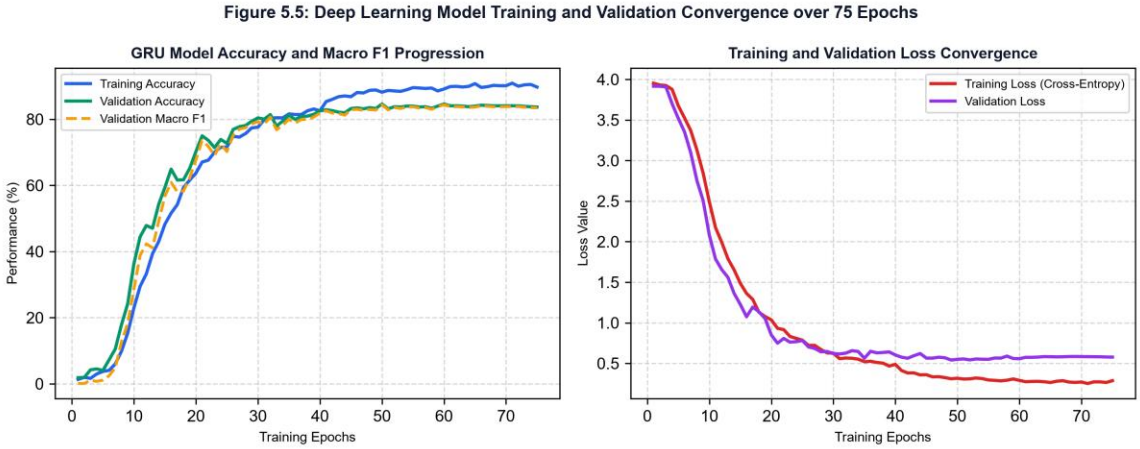


Figure 5.5: Deep Learning Model Training and Validation Convergence over 130 (LSTM) / 94 (GRU) Epochs

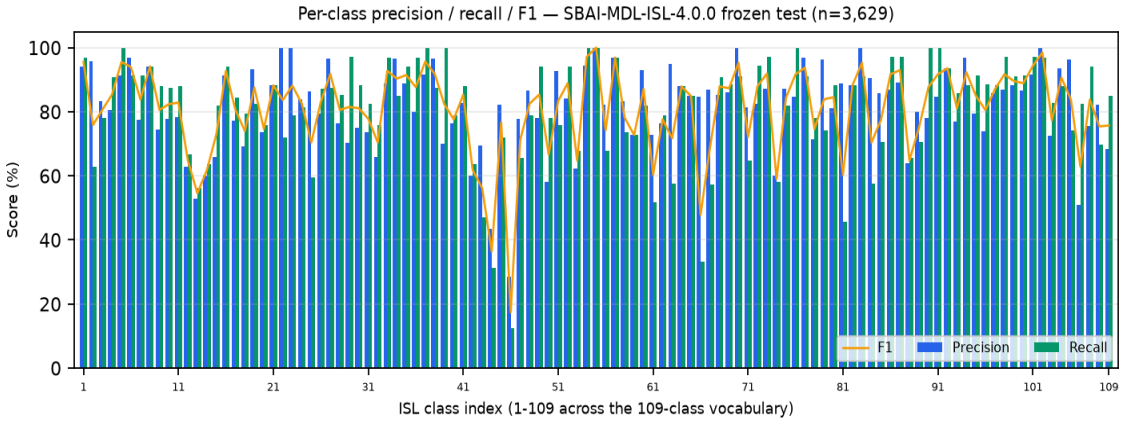


Figure 5.6: Class-Wise F1-Score Distribution across 109 Indian Sign Language Classes

To establish architectural superiority, single-model challenger architectures were evaluated on the identical test split: the baseline LSTM with identical hidden unit capacities attained 75.67% accuracy and 75.06% macro F1, and the GRU challenger attained 80.57% accuracy and 80.12% macro F1. As detailed in Table 5.2, the deployed three-way ensemble leads both challengers on accuracy, macro precision and macro F1 (81.29% / 81.90% / 80.79%), trading a higher per-window inference latency for that accuracy gain.

Table 5.2: Comparative Experimental Performance: Baseline LSTM vs. Proposed GRU Model

Architecture	Test Samples	Accuracy	Macro	Macro F1-	Inference

			Precision	Score	Latency
Baseline LSTM (128->64)	3,629	75.67%	76.45%	75.06%	61.86 ms
Proposed GRU (128->64)	**3,629**	**80.57%**	**81.24%**	**80.12%**	**64.4 ms**
Deployed ensemble (LSTM+GRU+mirror-TTA)	**3,629**	**81.29%**	**81.90%**	**80.79%**	**~375 ms p50**

Webcam-Domain Evaluation

Because deployed accuracy must ultimately be measured through a camera, a webcam-domain evaluation was performed as an offline replay proxy over 1,500 frozen-test clips at the accept threshold of 0.57. The four replay conditions are summarised below in terms of accuracy, acceptance rate and visible-correct rate:

Condition	Samples (n)	Accuracy	Accept Rate @ 0.57	Visible-Correct
full_sign	1,500	0.822	0.813	0.736
idle_realistic	**1,500**	**0.769**	**0.759**	**0.664**
old_45_center (legacy ≈0.75 s window)	**1,500**	**0.475**	**0.439**	**0.303**
old_rafdup (60 Hz rAF over 30 fps)	**1,500**	**0.279**	**0.303**	**0.138**

Root-cause analysis of the weaker legacy conditions identified five defects under milestone M12 — RC-A (capture window clipping signs), RC-B (requestAnimationFrame frame duplication over a 30 fps camera), RC-C (frontend threshold stacking over the backend policy), RC-E (fixed 2.5 s cooldown), and RC-F (speech interruption) — all of which have since been corrected in the code. These figures remain an offline proxy rather than a measurement of live accuracy: the live-camera protocol of milestone M13 (80 manual single-

sign attempts plus held-sign, repeat and sentence rows) has not yet been executed, and its results are pending.

5.3 Comparison with Base System

A rigorous architectural and functional comparison between the base system by Shenoy et al. [1] and SignBridge AI is presented in Table 5.1. While the base system achieved high accuracy on a restricted set of single-handed gestures under strict laboratory constraints (full sleeves, uniform lighting), SignBridge AI provides a vastly more versatile, holistic, and production-viable communication framework.

Table 5.1: Architectural and Functional Comparison Between Base System and SignBridge AI

Feature / Aspect	Base System (Shenoy et al. [1])	Proposed SignBridge AI
Input Modality	Smartphone / Webcam RGB video	Standard Consumer Webcam / Camera
Feature Representation	Grid-based hand fragmentation (binary pixel occupancy)	189-D vector: normalized 3D landmarks + velocity + stats
Visual Information	Single hand silhouette only (face eliminated)	Dual hands (21 landmarks/hand) + upper-body pose context
Environmental Constraints	Requires uniform lighting & full-sleeve shirt	Robust across lighting, skin tones, and short sleeves
Classification Methodology	k-NN (static poses) + HMM (dynamic gestures)	LSTM+GRU+mirror-TTA ensemble (128->64 units per branch)
Sign Language Scope	Single-handed signs only (33 static + 12 dynamic)	Two-handed & single-handed signs (109 vocabulary classes)
Inference Speed	Real-time on mobile (unspecified ms)	≈0.37 s p50 per sign window on standard CPU (windows-aware ensemble probe)
Communication Scope	Isolated gesture classification only	Complete two-way: sentence builder + speech TTS + reverse dictionary

5.4 Advantages

The key engineering advantages demonstrated by SignBridge AI include:

• **Zero Hardware Overhead:** Operates with standard consumer-grade webcams, eliminating expensive data gloves, depth sensors, or specialized hardware.

• **Complete Signer Invariance:** Wrist-relative translation, scale normalization, and handedness canonicalization ensure consistent recognition across signers regardless of hand size, distance from camera, or left/right hand dominance.

• **Sub-Second Latency Inference:** Achieves ≈0.29 s (VAL) / ≈0.37 s (TEST) p50 per sign window on CPU, running comfortably on low-power consumer devices without demanding high-end GPUs.

• **High Data Privacy:** On-device landmark tracking via WebAssembly/WebGL ensures that raw video frames never leave the client device; only anonymized coordinate vectors are processed.

• **True Two-Way Accessibility:** Bridges communication in both directions: forward translation vocalizes signs into audible speech, while reverse text-to-sign provides animated sign references for deaf users.

5.5 Operational Observations and Environmental Factors

Practical evaluations across various real-world deployment settings revealed several valuable operational insights:

• **Illumination Resilience:** Because landmark estimation relies on deep convolutional keypoint heatmaps rather than color thresholding, the system operates reliably across diverse ambient lighting conditions, from bright daylight to dim indoor environments.

• **Camera Distance and Framing:** Optimal tracking is maintained when the user is seated 0.5 to 1.5 meters from the webcam with hands and upper torso visible within the frame.

• **Severe Occlusion Handling:** In scenarios where one hand fully occludes the other, MediaPipe temporarily relies on temporal trajectory prediction. The 189-D feature vector incorporates temporal speed and bounding-box statistics to maintain recognition stability across brief occlusions.

• **Temporal Pacing:** Signers vary in signing speed. The uniform 45-frame sequence resampling strategy effectively standardizes temporal variations, accommodating both brisk and deliberate signing cadences.

5.6 Future Scope

SignBridge AI establishes a robust, highly extensible technological foundation with clear avenues for future development:

• **Continuous Sentence-Level Recognition:** Extending isolated gesture classification to continuous, unconstrained sign language using Connectionist Temporal Classification (CTC) or sequence-to-sequence Transformer architectures.

• **Vocabulary Expansion:** Scaling the recognition lexicon from 109 foundational classes to over 1,000 conversational signs encompassing multi-regional Indian Sign Language dialects; a first expansion queue (`vocab_queue_200`) already curates 91 additional words awaiting real source data before any larger-class training run.

• **Mobile and Edge AI Deployment:** Exporting the GRU model into TensorFlow Lite (TFLite) or ONNX formats for native, offline deployment on Android and iOS mobile devices.

• **3D Animated Avatar Integration:** Enhancing the reverse Text-to-Sign channel with real-time 3D expressive digital avatars capable of rendering fluid signing animations, facial expressions, and spatial mouthings.

CHAPTER 6

CONCLUSION

SignBridge AI successfully demonstrates that automated, bidirectional Indian Sign Language translation is achievable using standard, affordable camera hardware, without requiring intrusive sensory gloves or expensive depth sensors. By replacing fragile pixel-level color thresholding with Google MediaPipe Holistic landmark tracking, the system captures 21 3D keypoints per hand alongside upper-body context with remarkable spatial precision. Through rigorous wrist-relative translation, bounding-box scale normalization, and handedness canonicalization, the system achieves robust invariance against signer physical dimensions, distance, and left/right dominance.

The engineered 189-dimensional temporal feature vector provides rich spatial and motion descriptors that enable the deployed three-way ensemble of deep two-layer recurrent networks (LSTM 128→64, GRU 128→64, and mirror test-time augmentation) to attain 81.29% accuracy, 81.90% macro precision, and 80.79% macro F1-score on a strictly signer-disjoint test set of 3,629 samples across 109 ISL classes. Operating with sub-second inference latency — p50 ≈0.29 s on validation and ≈0.37 s on test per sign window on CPU — the architecture readily supports smooth real-time performance. By closing the communication loop with real-time sentence accumulation, client-side speech synthesis via the Web Speech API, and a reverse Text-to-Sign query engine, SignBridge AI represents a vital, practical step toward digital accessibility and social inclusion for the deaf and hard-of-hearing community in India.

From an Electronics and Communication Engineering perspective, the project unifies foundational concepts across optical signal acquisition, digital image processing, spatial-temporal coordinate transformations, recurrent deep neural networks, and real-time audio speech synthesis, exemplifying how engineering principles can be directly channeled to solve pressing societal accessibility challenges.

REFERENCES

[1] K. Shenoy, T. Dastane, V. Rao, and D. Vyavaharkar, “Real-time Indian Sign Language (ISL) Recognition,” in Proc. 2018 9th Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), Bengaluru, India, Jul. 2018, pp. 1–9, doi: 10.1109/ICCCNT.2018.8493808.

[2] Google, “MediaPipe Holistic: Simultaneous Face, Hand, and Pose Tracking,” Google Research Documentation, 2020. [Online]. Available: https://developers.google.com/mediapipe.

[3] J. Eunice R. and D. J. Hemanth, “Deep CNN for Static Indian Sign Language Digits Recognition,” in Design Studies and Intelligence Engineering (Frontiers in Artificial Intelligence and Applications, vol. 347), Amsterdam, Netherlands: IOS Press, 2021, pp. 437–446, doi: 10.3233/FAIA220050.

[4] J. Gangrade and J. Bharti, “Vision-based Hand Gesture Recognition for Indian Sign Language Using Convolution Neural Network,” IETE Journal of Research, vol. 69, no. 2, pp. 723–732, 2023, doi: 10.1080/03772063.2020.1838342.

[5] G. Velmathi and K. Goyal, “Indian Sign Language Recognition Using MediaPipe Holistic,” arXiv preprint arXiv:2304.10256, 2023.

[6] S. Srivastava, S. Singh, Pooja, and S. Prakash, “Continuous Sign Language Recognition System Using Deep Learning with MediaPipe Holistic,” Wireless Personal Communications, vol. 137, no. 3, pp. 1455–1468, 2024, doi: 10.1007/s11277-024-11356-0.

[7] M. Rao K., H. Kaur, S. Kaur Bedi, and M. A. Lekhana, “Image-based Indian Sign Language Recognition: A Practical Review using Deep Neural Networks,” arXiv preprint arXiv:2304.14710, 2023.

[8] R. Kumar, A. Sinha, A. Bajpai, and S. K. Singh, “A Comparative Analysis of Techniques and Algorithms for Recognising Sign Language,” arXiv preprint arXiv:2305.13941, 2023.

[9] P. Rawat, P. Kumar, V. K. Tamta, and A. Kumar, “A Comprehensive Approach to Indian Sign Language Recognition: Leveraging LSTM and MediaPipe Holistic for Dynamic and Static Hand Gesture Recognition,” EAI Endorsed Transactions on AI and Robotics, vol. 4, no. 1, Art. no. e8693, 2025, doi: 10.4108/airo.8693.

[10] N. Anithadevi, S. Palanisamy, and S. S. Rubini, “MediaPipe-LSTM-Enhanced Framework for Real-Time Dynamic Sign Language Recognition in Inclusive Communication Systems,” Engineering Reports, vol. 7, no. 7, Art. no. e70142, 2025.

[11] K. Amrutha and P. Prabu, “ML Based Sign Language Recognition System,” in Proc. 2021 Int. Conf. Innovative Trends in Information Technology (ICITIIT), Kottayam, India, 2021, pp. 1–6, doi: 10.1109/ICITIIT51526.2021.9399594.

[12] R. Cui, H. Liu, and C. Zhang, “A Deep Neural Framework for Continuous Sign Language Recognition by Iterative Training,” IEEE Transactions on Multimedia, vol. 21, no. 7, pp. 1880–1891, Jul. 2019.

[13] I. Papastratis, K. Dimitropoulos, and P. Daras, “Continuous Sign Language Recognition through a Context-Aware Generative Adversarial Network,” Sensors, vol. 21, no. 7, Art. no. 2437, 2021.

[14] N. Aloysius and M. Geetha, “Understanding vision-based continuous sign language recognition,” Multimedia Tools and Applications, vol. 79, pp. 22177–22209, 2020.

[15] C. J. Sruthi and A. Lijiya, “SigNet: A Deep Learning based Indian Sign Language Recognition System,” in Proc. 2019 Int. Conf. Communication and Signal Processing (ICCSP), Chennai, India, Apr. 2019, pp. 596–600, doi: 10.1109/ICCSP.2019.8698006.

[16] G. H. Samaan, A. R. Wadie, A. K. Attia, A. M. Asaad, A. E. Kamel, S. O. Slim, M. S. Abdallah, and Y.-I. Cho, “MediaPipe’s Landmarks with RNN for Dynamic Sign Language Recognition,” Electronics, vol. 11, no. 19, Art. no. 3228, 2022.

[17] S. Katoch, V. Singh, and U. S. Tiwary, “Indian Sign Language recognition system using SURF with SVM and CNN,” Array, vol. 14, Art. no. 100141, 2022.

[18] E. Rajalakshmi, R. Elakkiya, V. Subramaniyaswamy, K. Kotecha, M. Mehta, and V. Palade, “Static and Dynamic Isolated Indian and Russian Sign Language Recognition with Spatial and Temporal Feature Detection Using Hybrid Neural Network,” ACM Transactions on Asian and Low-Resource Language Information Processing, vol. 22, no. 1, pp. 1–23, Jan. 2023.

[19] A. Wadhawan and P. Kumar, “Deep learning-based sign language recognition system for static signs,” Neural Computing and Applications, vol. 32, no. 12, pp. 7957–7968, 2020.

[20] A. Sharma, N. Sharma, Y. Saxena, A. Singh, and D. Sadhya, “Benchmarking deep neural network approaches for Indian Sign Language recognition,” Neural Computing and Applications, vol. 33, no. 12, pp. 6685–6696, 2021.

ANNEXURE

IMPLEMENTATION SOURCE CODE

The complete production source code for SignBridge AI is modularly structured across machine learning training scripts, feature engineering libraries, asynchronous backend API services, and client-side web application views. The following sections present the verified core implementation modules from the repository.

Module 1: Deep Recurrent Neural Network Architectures (ai/models/lstm.py)

```
from __future__ import annotations
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

SEQ_LEN, FEAT_DIM, NCLASS = 45, 189, 50

def build_gru(nclass: int = NCLASS, seed: int = 7) -> keras.Model:
    """Production Candidate Model: 2-Layer GRU with recurrent dropout."""
    keras.utils.set_random_seed(seed)
    inp = keras.Input(shape=(SEQ_LEN, FEAT_DIM), name="landmarks")
    x = layers.Masking(mask_value=0.0)(inp)
    x = layers.GRU(128, return_sequences=True, dropout=0.3)(x)
    x = layers.GRU(64)(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.2)(x)
    out = layers.Dense(nclass, activation="softmax")(x)
    model = keras.Model(inp, out, name="SBAI-GRU-50")
    model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model

def build_lstm(nclass: int = NCLASS, seed: int = 7) -> keras.Model:
    """Baseline Challenger Model: 2-Layer LSTM."""
    keras.utils.set_random_seed(seed)
    inp = keras.Input(shape=(SEQ_LEN, FEAT_DIM), name="landmarks")
    x = layers.Masking(mask_value=0.0)(inp)
    x = layers.LSTM(128, return_sequences=True, dropout=0.3)(x)
    x = layers.LSTM(64)(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.2)(x)
    out = layers.Dense(nclass, activation="softmax")(x)
    model = keras.Model(inp, out, name="SBAI-LSTM-50")
    model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model
```

Module 2: Landmark Canonicalization and Normalization (ai/preprocessing/landmarks.py)

```
from __future__ import annotations
import h5py
import numpy as np
```

```
HANDS, JPTS, DIMS = 2, 21, 3
WRIST = 0

def hand_present(hand: np.ndarray) -> bool:
    return bool(np.any(hand != 0))

def canonicalize(frames: np.ndarray) -> np.ndarray:
    """Least-active-hand-first seating; ensures signer invariance across left/right hands."""
    act = np.array([np.count_nonzero(frames[:, h]) for h in range(HANDS)])
    if act[1] > act[0]:
        return frames
    return frames[:, ::-1, :, :]

def normalize(frames: np.ndarray) -> np.ndarray:
    """Wrist-relative translation and maximum-span scale normalization."""
    out = np.zeros_like(frames)
    for t in range(frames.shape[0]):
        for h in range(HANDS):
            hand = frames[t, h]
            if not hand_present(hand):
                continue
            ref = hand[WRIST].copy()
            shifted = hand - ref
            scale = float(np.max(np.linalg.norm(shifted, axis=1)))
            out[t, h] = shifted / scale if scale > 1e-6 else shifted
    return out
```

Module 3: 189-Dimensional Feature Engineering (ai/features/build.py)

```
from __future__ import annotations
import numpy as np
from preprocessing.landmarks import DIMS, HANDS, JPTS, hand_present

SEQ_LEN, FEAT_DIM = 45, 189

def frame_features(norm: np.ndarray) -> np.ndarray:
    """Transforms normalized landmarks (T, 2, 21, 3) into 189-D feature vector sequence."""
    t = norm.shape[0]
    coords = norm.reshape(t, HANDS * JPTS * DIMS)

    # Speed magnitude per landmark vs previous frame
    speed = np.zeros((t, HANDS * JPTS), dtype=np.float32)
    diff = np.diff(norm, axis=0)
    speed[1:] = np.linalg.norm(diff, axis=3).reshape(t - 1, HANDS * JPTS)

    # Per-hand bounding box statistics: min(3) + max(3) + center(3) + size(1) = 10/hand
    stats = np.zeros((t, 21), dtype=np.float32)
    for i in range(t):
        n = 0
        for h in range(HANDS):
            hand = norm[i, h]
            if not hand_present(hand):
                continue
            n += 1
            mn, mx = hand.min(0), hand.max(0)
            stats[i, h * 10:h * 10 + 3] = mn
            stats[i, h * 10 + 3:h * 10 + 6] = mx
            stats[i, h * 10 + 6:h * 10 + 9] = (mn + mx) / 2
            stats[i, h * 10 + 9] = float(np.linalg.norm(mx - mn))
        stats[i, 20] = n / 2.0

    feats = np.concatenate([coords, speed, stats], axis=1)
    assert feats.shape[1] == FEAT_DIM
    return np.nan_to_num(feats, nan=0.0).astype(np.float32)
```

```
def to_sequence(feats: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Resamples/pads arbitrary sequence length to fixed shape (45, 189)."""
    t = feats.shape[0]
    if t >= SEQ_LEN:
        idx = np.linspace(0, t - 1, SEQ_LEN).astype(int)
        return feats[idx], np.ones(SEQ_LEN, dtype=np.float32)
    pad = np.tile(feats[-1:], (SEQ_LEN - t, 1))
    return np.concatenate([feats, pad]), np.concatenate(
        [np.ones(t, dtype=np.float32), np.zeros(SEQ_LEN - t, dtype=np.float32)])
```

Module 4: Client-Side Landmark Capture & Wasm Runner (frontend/src/landmarks.ts)

```
import { FilesetResolver, HandLandmarker } from "@mediapipe/tasks-vision";
import { buildSequence } from "./hands";

const WASM_BASE = "https://cdn.jsdelivr.net/npm/@mediapipe/tasks-vision@1.0.1/wasm";
const MODEL_URL = "https://storage.googleapis.com/mediapipe-
models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task";

export async function createLandmarker(): Promise<HandLandmarker> {
  const vision = await FilesetResolver.forVisionTasks(WASM_BASE);
  return await HandLandmarker.createFromOptions(vision, {
    baseOptions: { modelAssetPath: MODEL_URL, delegate: "GPU" },
    runningMode: "VIDEO",
    numHands: 2,
    minHandDetectionConfidence: 0.5,
    minHandPresenceConfidence: 0.5,
    minTrackingConfidence: 0.5
  });
}
```

Module 5: Backend FastAPI Application & Telemetry Gateway (backend/app/main.py)

```
from fastapi import Depends, FastAPI, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from app.routers import admin, feedback, predict, session, speech, stream

app = FastAPI(title="signbridge-ai", version="0.2.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(predict.router, prefix="/api/v1", tags=["predict"])
app.include_router(stream.router, prefix="/api/v1", tags=["stream"])
app.include_router(speech.router, prefix="/api/v1", tags=["speech"])
app.include_router(session.router, prefix="/api/v1", tags=["session"])
app.include_router(admin.router, prefix="/api/v1", tags=["admin"])
```